

Features - Dominion Arena Tatica

Features Implementadas

Backend (Node.js + Express + Socket.io)

Servidor

- [x] Express.js para rotas HTTP
- [x] Socket.io para comunicacao em tempo real
- [x] CORS configurado
- [x] Rotas REST basicas

Banco de Dados

- [x] Integracao PostgreSQL
- [x] Schema de tabelas (games, players, game_states)
- [x] Health check
- [x] Inicializacao automatica

Logica de Jogo (GameLogic.js)

- [x] Sistema de turnos (roll, buy, combat, end)
- [x] Sistema de dados (1-3 dados, rolar para gerar ouro)
- [x] Loja dinamica (6 cartas aleatorias)
- [x] Compra de cartas (com validacao de ouro)
- [x] Sistema de evolucao (2 iguais = 1 evoluida)
- [x] Calculo de combate (ATK - DEF)
- [x] Calculo de sinergias (por regio e classe)
- [x] Posicionamento de cartas no campo
- [x] Verificacao de limites (mao, campo, dados)

WebSocket Events

- [x] join-game
- [x] roll-dice
- [x] generate-shop
- [x] buy-card
- [x] place-card
- [x] check-evolution
- [x] calculate-synergies
- [x] next-phase
- [x] attack-player
- [x] disconnect handling

Frontend (Next.js + React)

Paginas

- [x] Pagina inicial / Menu
- [x] Criacao de sala

- [x] Entrada em sala
- [x] Tela de jogo

Componentes React

- [x] GameBoard - Componente principal
- [x] PlayerStats - Estatísticas do jogador
- [x] Shop - Loja de cartas
- [x] Card - Componente de carta
- [x] Hand - Mao de cartas
- [x] Field - Campo de batalha
- [x] GamePhase - Status da fase
- [x] Synergies - Sinergias ativas
- [x] Opponents - Lista de opositores

Utilidades

- [x] Socket.io client
- [x] API client
- [x] Gerenciamento de estado com React hooks

Estilos

- [x] CSS global
- [x] Design responsivo
- [x] Tema escuro
- [x] Sem animacoes (CSS basico)

Dados

Cartas (40 cartas)

- [x] 7 Classes: Guerreiro, Mago, Ladino, Suporte, Monstro, Mercador, Dragao
- [x] 6 Regioes: Floresta, Vulcao, Montanha, Lago, Ceu, Deserto
- [x] Custos de 2 a 8 ouro
- [x] ATK e DEF variados
- [x] Habilidades descritivas

Mecanicas

Inicio

- [x] 20 vidas por jogador
- [x] 1 dado inicial
- [x] Maximo 3 dados
- [x] Limite de 5 cartas na mao
- [x] Limite de 6 cartas no campo

Combate

- [x] Calculo automatico
- [x] Dano = $\text{Max}(0, \text{ATK} - \text{DEF})$
- [x] Reduz vidas

Evolucao

- [x] 2 cartas iguais = 1 evoluida

- [x] +30% ATK e DEF

Sinergias

- [x] Contagem por regioao
- [x] Contagem por classe
- [x] Display de sinergias ativas

Features Nao Implementadas (Futuro)

- [] Animacoes e transicoes
- [] Som e musica
- [] Sistema de ranking
- [] Historico de partidas
- [] Editor de decks
- [] Modo single-player com IA
- [] Eventos especiais
- [] Itens/Artefatos
- [] Chat in-game
- [] Sistema de torneios
- [] Perks permanentes
- [] Premium skins para cartas
- [] Loja in-game com skins

Endpoints REST

- GET /api/health - Verifica saude do servidor
- POST /api/games - Cria nova sala
- GET /api/games/:gameId - Obtem estado da sala

Eventos WebSocket

Cliente -> Servidor

- join-game
- roll-dice
- generate-shop
- buy-card
- place-card
- check-evolution
- calculate-synergies
- next-phase
- attack-player

Servidor -> Cliente

- game-state
- player-joined
- player-left

- dice-rolled
- shop-generated
- card-bought
- card-placed
- evolution-checked
- phase-changed
- synergies-calculated
- combat-resolved
- error

Configuracoes

Variaveis de Ambiente

Backend (.env):

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/dominion_arena
PORT=5000
NODE_ENV=development
FRONTEND_URL=http://localhost:3000
```

Frontend (.env.local):

```
NEXT_PUBLIC_API_URL=http://localhost:5000
NEXT_PUBLIC_SOCKET_URL=http://localhost:5000
```

Dependencias Principais

Backend

- express: Framework HTTP
- socket.io: WebSocket real-time
- pg: PostgreSQL driver
- cors: Cross-origin requests
- uuid: Geracao de IDs

Frontend

- next: Framework React
- react: UI library
- socket.io-client: WebSocket client
- react-dom: React rendering

Arquivos Principais

```

dominion_arena_tatica/
├── backend/
│   ├── server.js           # Servidor principal
│   ├── gameLogic.js       # Logica de jogo
│   ├── database.js        # PostgreSQL
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── pages/         # Paginas Next.js
│   │   ├── components/    # Componentes React
│   │   ├── lib/           # Utilidades
│   │   └── styles/        # CSS
│   └── package.json
├── dominion_cards.json     # Banco de 40 cartas
├── docker-compose.yml     # PostgreSQL com Docker
├── validate.sh            # Script de validacao
├── start.sh               # Script de inicializacao
├── README.md              # Documentacao principal
├── SETUP.md               # Guia de instalacao
├── MECANICAS.md           # Mecanicas de jogo
├── DESENVOLVIMENTO.md    # Guia de desenvolvimento
└── FEATURES.md            # Este arquivo

```

Status do Projeto

PRONTO PARA USAR!

Todos os arquivos foram criados e a estrutura foi validada.

Para iniciar:

1. Instalar Node.js e PostgreSQL
2. `cd backend && npm install`
3. `cd ../frontend && npm install`
4. `docker-compose up -d` (para PostgreSQL com Docker)
5. `npm run dev` (em cada pasta)